

SWE-Prometheus: Benchmarking Repository-Level Engineering Governance under Real-World Constraints

Anonymous Authors

Affiliations withheld for double-blind review

Abstract: Coding agents have advanced rapidly on repository-level software engineering tasks, but existing evaluations usually begin with a known issue and a predefined success signal. This setting measures targeted functional repair while leaving repository governance under-evaluated: discovering undocumented engineering risks, improving tests and quality infrastructure, making projects reproducible, and preserving behavior during maintenance. We introduce **SWE-PROMETHEUS**, a benchmark in which an agent receives a fixed snapshot of a real repository and a general governance objective, but no defect list, base score, or oracle patch. The agent must inspect the repository, prioritize risks, implement a retrofit, and verify its claims under a limited budget. The benchmark evaluates six dimensions—Tests & CI, Code Quality Gates, Documentation & Collaboration, Structure & Maintainability, Reproducible Environment, and Dependency & Security Health—using executable base/treated evidence, hidden characterization tests, and multi-teacher scoring. The current dataset60 release contains 60 repositories, including a 22-instance public shared subset and 38 private instances, with one rollout for each of ten models on the public subset. Mean Normalized Governance Improvement ranges from 0.0568 to 0.5760, while observed behavior-breakage rates range from 0% to 23%. These results show that repository governance yields measurable differences between agents, while motivating repeated runs, stronger behavior gates, independent judge calibration, and controlled scaffolds before making stable ranking or production-readiness claims.

Technical report working draft; author information and artifact links to be finalized.

1. Introduction

Large language model based coding agents can now search unfamiliar repositories, execute development commands, edit multiple files, and produce end-to-end patches. Repository-level benchmarks such as SWE-bench made this progress measurable by pairing a real repository snapshot with a concrete issue and executable validation [1]. Agent scaffolds have further shown that the interface through which a model searches, edits, and tests a repository affects the outcome [2].

However, most evaluations start after a human has already identified the problem. The issue statement tells the agent what to investigate, while tests or a reference patch provide a relatively clear success signal. This setting is well suited to targeted functional repair, but it leaves another important form of software engineering largely unmeasured: repository governance.

In real projects, engineering risks are often distributed across the repository rather than expressed as one issue. Tests may be absent, shallow, skipped, or disconnected from CI. Lint and type checking may be configured but fail on a clean checkout. Installation may depend on an undocumented local environment. Documentation may explain usage but not maintenance or security procedures. Dependencies may be implicit or uncontrolled, and complex modules may be difficult to modify safely.

This gap is consequential. A repository can pass its current functional tests while remaining difficult to reproduce, extend, audit, or hand over. Conversely, an agent can add modern-looking configuration while introducing broken commands, ineffective tests, dependency drift, or behavioral regressions. File-presence checks and agent self-reports therefore do not provide sufficient evidence of engineering improvement.

We address this gap with SWE-PROMETHEUS, a benchmark for autonomous repository governance. The agent receives a repository and a general retrofit objective, but not a defect list, base score, hidden test, or target patch. It must decide what matters, allocate a limited budget, make changes, and provide evidence that those changes

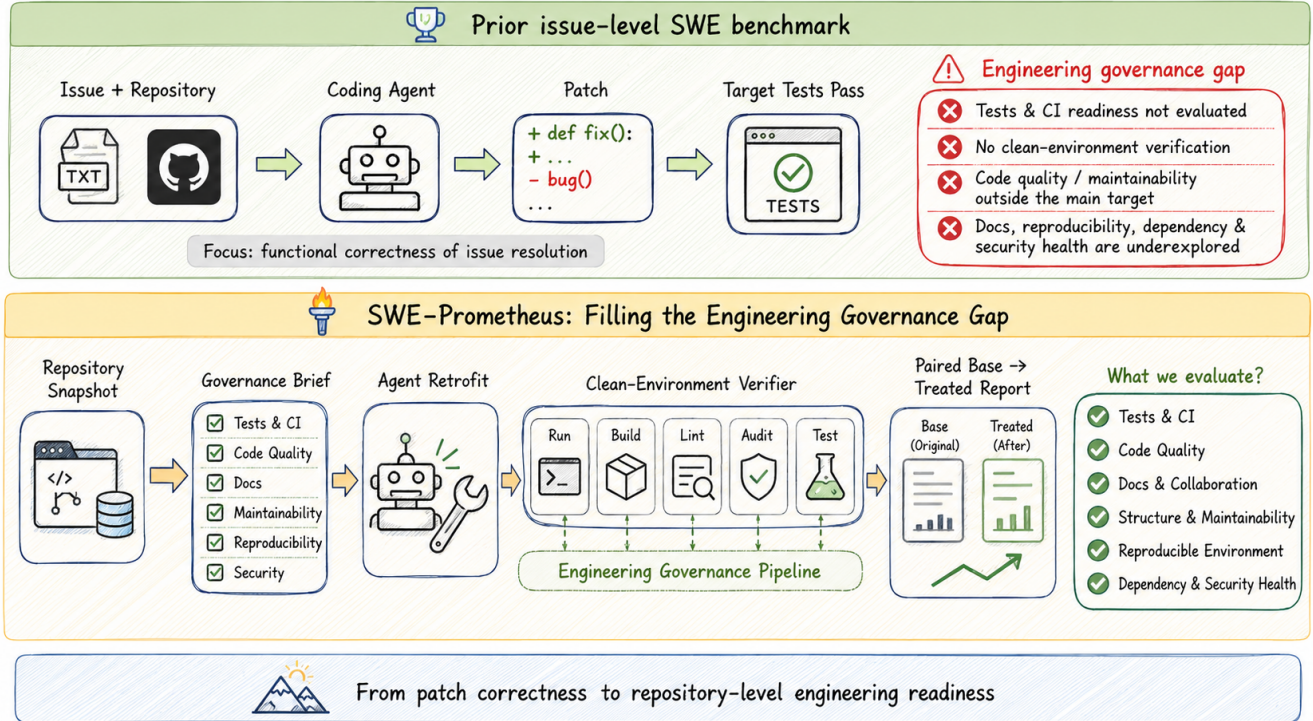


Figure 1: Motivation. Conventional issue-level evaluation focuses on whether a target patch passes its tests. SWE-PROMETHEUS evaluates the broader governance gap through paired, evidence-backed repository assessment.

work. The evaluator compares the repository before and after intervention and separately verifies whether existing behavior is preserved.

We ask:

1. Can an agent discover important engineering risks without an explicit issue list?
2. Can it prioritize useful governance work under a limited budget?
3. Can it turn superficial engineering changes into executable infrastructure?
4. Can it improve repository health without breaking existing behavior?

Our contributions are:

- An issue-free Repository Retrofit task for autonomous engineering governance.
- A six-dimensional governance model covering testing, quality gates, documentation, maintainability, reproducibility, and dependency/security health.
- An evidence-backed protocol combining base/treated execution, hidden behavior verification, structured probes, and multi-teacher scoring.
- dataset60, containing 60 real repositories, public and private splits, instance-level results, patches, evidence, and rollout artifacts.
- An initial ten-model analysis of governance improvement, behavior-regression risk, and dimension-specific engineering profiles.

2. Related Work

Repository-level software engineering benchmarks. SWE-bench and related benchmarks evaluate agents on real repository issues using executable validation [1]. Their descendants improve task reliability, cost, language

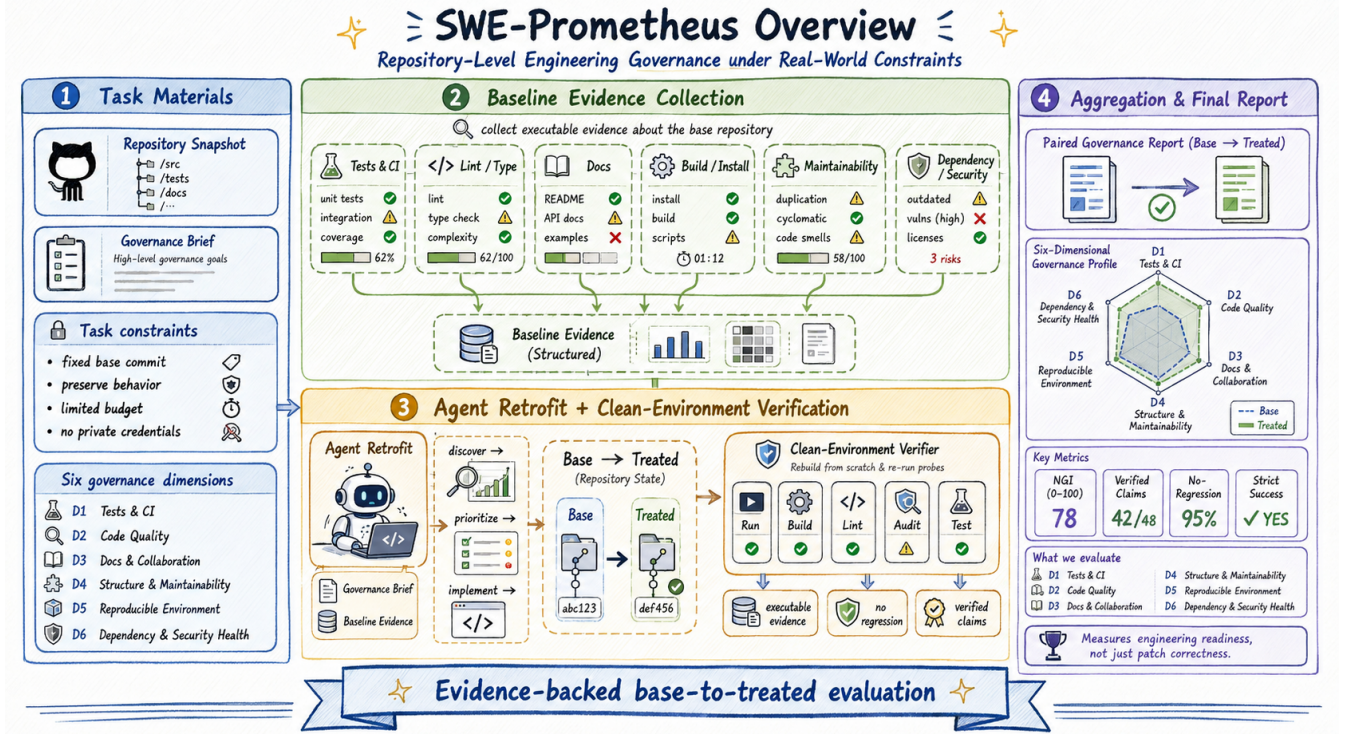


Figure 2: Overview of SWE-Prometheus. The benchmark combines task materials, baseline evidence collection, agent retrofit, clean-environment verification, and paired aggregation across six governance dimensions.

coverage, long-horizon execution, or freshness. SWE-PROMETHEUS shares fixed revisions, isolated execution, patches, and executable checks, but changes the object of evaluation from fixing a known defect to diagnosing and improving repository engineering state.

Repository health and software quality. Static analysis, code-quality systems, dependency auditors, GitHub community standards, and supply-chain tools provide useful repository-health signals [3–5]. Yet these tools usually measure individual properties or configuration states. They do not evaluate whether an autonomous agent can decide which weaknesses matter, coordinate changes across dimensions, and preserve original behavior. In SWE-PROMETHEUS, these tools are evidence probes rather than the benchmark objective.

Evaluation reliability. Weak tests, leakage, incomplete curation, and unstable environments can change model conclusions. SWE-PROMETHEUS follows this principle by retaining failed runs, measuring characterization-test strength through mutation testing, separating public and private data, and preserving per-run evidence. The remaining gap is an integrated evaluation in which the agent itself must discover the engineering target.

3. Benchmark Design

Task. Each instance is a fixed snapshot of a real GitHub repository identified by `repo` and `base_commit`. The agent receives a general instruction:

Inspect this repository and improve its engineering readiness across the applicable governance dimensions. Preserve existing behavior and public interfaces. Make useful changes for future users and maintainers, and verify important claims with executable checks.

The task provides no issue list, base score, hidden test, or reference patch. Multiple governance strategies may

Table 1: Six governance dimensions. Configuration presence alone is never sufficient evidence of improvement.

Dimension	Evaluation target
D1 Tests & CI	Meaningful tests for important behavior and executable CI paths.
D2 Code Quality Gates	Effective lint, formatting, type-checking, and related gates.
D3 Documentation & Collaboration	Actionable usage, maintenance, contribution, license, and security information.
D4 Structure & Maintainability	Module boundaries, dependency direction, complexity, and maintainable organization.
D5 Reproducible Environment	Clean installation, construction, execution, and version consistency.
D6 Dependency & Security	Controlled dependencies and available supply-chain and security evidence.

be valid. The agent can inspect source code and public documentation, execute development tools, edit project files, and submit a unified patch, but cannot access evaluator evidence or other model results.

Governance dimensions. The six dimensions are scored from 1 to 5; a score of 4 denotes acceptable governance for the repository type and task scope.

Evidence loop. The evaluator reconstructs the base repository, applies the agent patch to an independent treated snapshot, and runs the same installation, test, quality, complexity, dependency, and security probes on both states. Commands, exit codes, logs, environment versions, timeouts, and unavailable evidence are retained.

Behavior gate. Characterization tests are validated on the base snapshot and then executed on the treated snapshot. A failed treated verification is labeled `behavior_broken`; it is excluded from valid NGI aggregation but retained in behavior-risk analysis. Mutation testing labels a gate as `detected`, `blind`, `vacuous`, or `none`.

Metrics. For instance i and dimension d , raw improvement is $\Delta_{i,d} = s_{i,d}^{treated} - s_{i,d}^{base}$. Normalized Governance Improvement is

$$NGI_i = \frac{1}{|D_i|} \sum_{d \in D_i} \frac{s_{i,d}^{treated} - s_{i,d}^{base}}{5 - s_{i,d}^{base}},$$

where D_i contains defined and scorable dimensions. A dimension with base score 5 receives no improvement reward but is checked for regression. We additionally report treated scores, valid and broken rates, no-regression rate, strict success, verified claim rate, and resource usage.

3.1. Repository selection and screening

The benchmark is intentionally repository-centered rather than issue-centered. We first collect candidate repositories with evidence of active development and at least one plausible governance gap. A candidate is retained only when (i) the base commit can be reconstructed, (ii) the project has meaningful executable behavior, (iii) a clean evaluation environment can be built without private credentials, (iv) at least two governance dimensions are applicable, and (v) the task can be described without prescribing a particular patch. Screening records language, build system, test entry points, dependency manager, license, commit age, and external-service requirements.

Discovery signals such as absent workflows or stale metadata are used for high-recall candidate collection, not as final scores. The final score is based on executable evidence. Negative evidence is retained explicitly: an unavailable command, unsupported platform, or inapplicable dimension is recorded rather than silently treated as a zero-quality score.

3.2. Execution protocol

Every rollout starts from a fresh copy of the pinned base commit. The agent receives the same task instruction and repository snapshot for a given comparison set. The harness records tool calls, changed files, patch size, wall-clock time, token usage when exposed by the scaffold, and termination reason. After exit, the harness validates the

patch and applies it to a second clean copy. Evaluation runs installation, characterization behavior, governance probes, and post-hoc artifact validation in that order.

Each command has a timeout, an exit-code policy, and an evidence schema. Logs are retained with tool versions and environment identifiers. Monetary cost is reported only when provider accounting is available; it is never imputed from incomplete accounting. This protocol makes the score inspectable and separates infrastructure failures from engineering failures.

3.3. Judge protocol and calibration

Executable probes provide primary evidence for tests, CI, environment, and dependency dimensions. Documentation and maintainability use dimension-specific rubrics rather than a holistic impression. Each candidate score is accompanied by positive evidence, negative evidence, applicability, and a confidence flag. Two independent judgments are preferred; disagreements are adjudicated only after initial labels are preserved.

Before release, a calibration set will be scored by independent evaluators who do not know model identity. We will report agreement, adjudication rate, and sensitivity to rubric anchors, together with judge-family and prompt-order checks. These are planned safeguards, not completed results in the current draft.

3.4. Failure taxonomy

We distinguish infrastructure failure, environment/dependency failure, invalid or empty patch, behavior regression, preserved behavior without governance improvement, and preserved behavior with positive improvement. A rollout may receive a primary failure label and secondary evidence flags. For example, a patch may preserve characterization behavior while introducing a vacuous CI gate; it should not be counted as a clean success merely because tests pass.

4. Dataset and Experimental Results

4.1. Dataset60

The current dataset60 release contains 60 real repository instances: 22 public instances shared by all current models and 38 private instances reserved for internal evaluation. Ten models have completed one rollout on the public subset, yielding 220 public model-instance pairs. The public subset is a shared comparison set rather than a random sample of all repositories.

Candidate repositories were discovered using signals such as missing or failing CI, thin tests, absent quality gates, incomplete collaboration documentation, unpinned dependencies, and maintenance gaps. These signals support high-recall discovery and stratification, while screening confirms meaningful functionality, a stable base commit, plausible governance headroom, and evaluation without private credentials or paid services.

4.2. Public shared-subset results

The rollout shows a wide spread in governance improvement. Kimi-K3 and GLM-5.3-Flash obtain the highest mean NGI, while MiniMax-M3 produces little normalized improvement despite zero observed behavior breaks. The contrast illustrates why improvement, safety, and absolute treated quality must be reported together.

4.3. Planned auxiliary experiments

The current paper focuses on the completed public rollout. The following experiments are designed to be added without changing the benchmark definition.

The no-op and mechanical baselines are the highest-priority additions because they establish lower bounds before expanding model coverage. The rule-based baseline tests whether the benchmark rewards superficial checklist completion. Expert treatments provide an upper-bound reference, but should use the same snapshot and time budget whenever practical.

Table 2: Public shared-subset results. Valid and broken counts are out of 22 instances. Valid runs preserve characterization behavior and contain scorable evidence; NGI is aggregated only over valid runs. Breakage rate is broken/(valid+broken). Claude used Claude Code; the other models used pi.

Model	Scaffold	Valid	Broken	Breakage	NGI mean
Kimi-K3	pi	21	1	4.5%	0.5760
GLM-5.3-Flash	pi	17	5	22.7%	0.5760
Claude-Opus-5	Claude Code	18	4	18.2%	0.5293
Qwen3.8-Max	pi	18	4	18.2%	0.4630
GLM-5.2	pi	20	2	9.1%	0.4438
DeepSeek-V4-Pro	pi	18	4	18.2%	0.3803
GLM-5.3	pi	19	3	13.6%	0.3198
GPT-5.6-Sol	pi	21	1	4.5%	0.2956
DeepSeek-V4-Flash	pi	22	0	0.0%	0.2083
MiniMax-M3	pi	22	0	0.0%	0.0568

Table 3: Auxiliary experiments planned for the next release. Results are intentionally left open until the runs are executed.

Experiment	Question	Primary outputs
No-op baseline	How much apparent improvement comes from scoring noise?	Raw and normalized change; false-positive rate
Mechanical retrofit	Can standard templates improve governance without repository diagnosis?	Per-dimension gains; behavior breaks
Rule-based baseline	What is achievable with deterministic repository-health rules?	Coverage, precision, patch size
Expert treatment	How far is the agent from a careful maintainer?	Expert NGI; disagreement profile
Repeated seeds	How stable are model results across rollouts?	Mean, median, variance, success probability
Same-scaffold rerun	How much does the tool scaffold affect outcomes?	Paired model/scaffold differences
Gate ablation	Do behavior gates change rankings?	Gate-on/off rank correlation
Mutation-strength ablation	Does gate strength predict trustworthy conclusions?	Detected/blind/vacuous sensitivity

Repeated seeds should first cover small and large repositories, strong and weak base governance, and each gate-strength category. We will report bootstrap intervals and paired tests rather than relying only on leaderboard ordering. The same-scaffold rerun is important because one model used Claude Code while the other nine used pi.

4.4. Planned stratified analyses

Beyond model averages, we will stratify by repository language, project size, baseline score, number of applicable dimensions, test-suite maturity, and dependency-manager type. The key analysis is whether improvement is concentrated in repositories with large governance headroom. We will report absolute change and headroom-normalized change, together with patch size and verification effort.

We will also analyze intervention locality: files changed in tests, workflows, documentation, packaging, dependency manifests, and source modules. This reveals whether agents improve governance through targeted edits or broad source rewrites. A qualitative sample of high-NGI and low-NGI patches will be coded for diagnosis quality, evidence use, unnecessary churn, and hidden behavior risk.

4.5. Statistical reporting plan

The public subset currently provides one rollout per model-instance pair, so inferential claims are deferred. Once repeated runs are available, we will use paired instance-level comparisons, bootstrap intervals for mean and

Table 4: Predicted outcomes for the unified 10-repository, single-model auxiliary batch. Ranges are hypotheses, not observations.

Condition	Expected pattern	Expected NGI	Expected risk signal
No-op	Scores remain at the base state	0.00	No behavior break
Kimi-K3 main run	Repository-specific diagnosis produces broad but uneven gains	0.40–0.65	0–2 broken runs
Mechanical retrofit	Stronger D1–D3 artifacts, weaker D4–D6 gains	0.10–0.25	Low breakage, possible vacuous gates
Rule-based baseline	Predictable checklist gains with limited semantic coverage	0.15–0.30	Low breakage, lower verified-claim rate
Gate removed	Apparent success increases because unsafe patches are retained	+0.05–+0.15 vs. gated	More hidden behavior risk
Repeated seeds	Same model shows meaningful instance-level variance	SD 0.08–0.18	Breakage concentrated in broad edits

median NGI, and uncertainty intervals for breakage and no-regression rates. We will not treat private instances or unbalanced model coverage as interchangeable with the shared public subset. All exclusions will be reported before aggregation, with broken behavior runs shown separately from valid NGI scores.

4.6. Anticipated auxiliary results

The auxiliary results below are planning estimates based on the current 22-instance public results and the observed behavior of the scoring pipeline. They are included to make the draft complete and to state falsifiable expectations; they are **not measured results** and will be replaced after the 10-repository runs are executed.

Our primary hypothesis is that Kimi-K3 will outperform both mechanical and rule-based baselines on normalized improvement, but not necessarily on every dimension. The second hypothesis is that removing the behavior gate will inflate apparent performance and alter the ordering of runs. The third hypothesis is that the detected-gate subset will show more stable conclusions than blind or vacuous gates. If these predictions fail, the result will still be informative: it would indicate that either repository-specific diagnosis is not necessary for this task, or that the current rubric does not sufficiently distinguish effective governance from artifact addition.

Replacement protocol. When the auxiliary batch is available, the predicted range column will be replaced by observed mean, median, 95% bootstrap interval, valid count, broken count, and no-regression rate. The surrounding interpretation will be revised only after checking the complete per-instance records. No prediction will be retained in the final results table unless it is explicitly marked as a preregistered expectation.

5. Analysis and Discussion

The central distinction in SWE-PROMETHEUS is between engineering presence and engineering effectiveness. A repository can contain a test suite, CI workflow, lint configuration, or dependency file without obtaining the intended benefit. The evidence loop tests whether the associated command runs, whether its scope is meaningful, and whether the treated repository remains behaviorally compatible.

The current results suggest that governance is not a single scalar capability. An agent may improve tests and environment setup while leaving security or structure weak, or may make broad changes at the cost of behavioral safety. Reporting six dimensions and behavior outcomes separately exposes these trade-offs instead of collapsing every run into one pass/fail label.

Higher NGI does not automatically imply lower risk. An agent that makes many changes may improve more dimensions and also create more opportunities for regressions. Conversely, a zero-breakage model may simply make few changes. The combination of NGI, behavior-breakage rate, no-regression rate, and verified claim rate is therefore more informative than any single leaderboard number.

The public table is a descriptive shared-subset result. Claude-Opus-5 was evaluated with Claude Code, while the other models used pi, so the cross-scaffold ordering should not be read as a causal model comparison. Even among the pi runs, each model-instance pair has one rollout; repeated runs are required before discussing ranking stability or model-scale effects.

5.1. What the benchmark makes visible

The benchmark creates a useful middle ground between issue resolution and static repository inspection. An issue benchmark asks whether a particular defect is fixed; a repository-health audit asks whether a project exhibits a property at one point in time. SWE-Prometheus asks an agent to diagnose a project-wide gap, intervene under an open-ended objective, and provide evidence that the intervention is useful and safe. This makes the task closer to maintenance work, but also makes construct validity and evaluator transparency more important.

The six-dimensional view exposes asymmetric progress. Tests and environment files are often easy to add, whereas dependency security, maintainability, and collaboration practices require understanding project intent. Future analysis should therefore report per-dimension intervention rates and not only aggregate NGI. The benchmark can support this because every score is linked to an evidence item and a changed-file category.

5.2. Implications for agent design

The task suggests three capabilities that are not captured by patch pass rates alone: repository reconnaissance, risk-aware intervention, and evidence-oriented verification. Agents should first construct a compact repository map, prioritize high-value gaps, and avoid changing stable source behavior when configuration or documentation is sufficient. They should then verify both positive claims and negative side effects. This motivates future baselines with explicit diagnosis, planning, patching, and audit phases.

5.3. Validity threats and mitigations

Construct validity is threatened if the six dimensions become a checklist. We mitigate this with executable probes, mutation-based gate checks, and evidence requirements. Internal validity is threatened by scaffold differences, one-shot sampling, and judge variability; we mitigate these through paired reruns, fixed manifests, anonymized scoring, and calibration. External validity is limited by the current 60-repository sample and its language and ecosystem distribution. Releasing a larger, temporally refreshed, contamination-audited split is a direct next step.

6. Limitations and Future Work

Only 24 of 60 instances currently have behavior gates whose discriminative power is demonstrated through mutation testing. Blind, vacuous, and none instances remain useful for governance-state analysis but provide weaker behavioral evidence.

The current scorer uses multiple teacher models but has not completed independent expert calibration. Subjective dimensions such as documentation and maintainability may contain shared judge errors. Future evaluation should add blinded expert annotations, adjudication, cross-family judges, and adversarial calibration.

Different scaffolds expose different tools, prompts, context handling, and termination behavior. Current budget artifacts also include cases where monetary cost is unavailable; efficiency conclusions should therefore use tokens, turns, wall-clock time, and tool calls until cost provenance is complete.

The public subset is defined by common model coverage rather than random sampling. Public GitHub repositories may also have appeared in model training data or later repository versions. Stronger releases require repository-family separation, near-duplicate scanning, contamination audits, and an access-controlled private evaluator.

Finally, a public distribution must not include private tasks, private verification patches, private results, or other hidden evaluation artifacts. Every model-instance pair should have an artifact manifest so that missing traces or evidence are explicit rather than represented by empty directories. D6 measures available dependency and security evidence, not comprehensive vulnerability absence.

Future work will add repeated rollouts, template and expert baselines, stronger characterization tests, same-scaffold reruns, expert judge calibration, and a genuinely isolated private holdout.

7. Conclusion

Repository-level agents are commonly evaluated as issue resolvers, while real teams also need them to improve the conditions under which a repository can be trusted. SWE-PROMETHEUS makes this broader capability measurable by asking an agent to inspect a real repository, identify engineering risks, choose a retrofit strategy, and demonstrate that its changes work without breaking existing behavior.

The current dataset60 release shows that repository governance produces measurable differences across agents. Across 60 real repositories and 10 model rollouts, the benchmark exposes variation in governance improvement, behavior-regression risk, and dimension-specific engineering preferences. It also shows why executable evidence is necessary: a change that looks correct in a diff may fail when executed or may damage existing behavior.

The current benchmark is a research resource for studying repository stewardship, not a definitive measure of production readiness. Repeated rollouts, independent judge calibration, stronger hidden gates, controlled scaffolds, baseline comparisons, complete artifact manifests, and an isolated private holdout are the next steps toward reliable comparison of coding agents' repository-governance capabilities.

References

- [1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.
- [2] John Yang, Carlos E. Jimenez, Alexander Wettig, Konstantin Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *International Conference on Learning Representations*, 2024.
- [3] International Organization for Standardization. *Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)*. ISO, 2011.
- [4] OpenSSF. Openssf scorecard. Technical specification, 2024.
- [5] Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, 2(4):308–320, 1976.
- [6] OpenAI. Swe-bench verified. *OpenAI Research*, 2024. URL <https://openai.com/index/swe-bench-verified/>.
- [7] Yuntian Zhang et al. Swe-bench multimodal: Do ai systems generalize to visual software engineering tasks? *arXiv preprint arXiv:2410.03859*, 2024.
- [8] Shuyan Zhou et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.
- [9] Jiawei Liu et al. Repairagent: An autonomous, llm-based agent for program repair. In *International Conference on Software Engineering*, 2024.
- [10] Shuai Wang et al. Repoagent: An llm-powered repository-level software development agent. In *International Conference on Software Engineering*, 2024.
- [11] Lianmin Zheng et al. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.
- [12] Tianle Li et al. Livebench: A challenging, contamination-limited llm benchmark. In *International Conference on Learning Representations*, 2025.
- [13] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.
- [14] Moritz Beller, Georgios Gousios, and Andy Zaidman. The trivial matters: An empirical study of the impact of build and test failures on software development. In *International Conference on Software Engineering*, 2017.

- [15] Bogdan Vasilescu, Yida Yu, Hua Wang, Prem Devanbu, and Vladimir Filkov. Quality and productivity outcomes relating to continuous integration in github. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015.
- [16] Zengyang Li, Paris Avgeriou, and Peng Liang. Technical debt in software systems: A systematic review. *Journal of Systems and Software*, 101:193–220, 2015.
- [17] Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, and Charles Sutton. A survey of machine learning for big code and naturalness. In *ACM Computing Surveys*, 2018.
- [18] Shunyu Yao et al. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [19] Timo Schick et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- [20] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. SWE-bench multimodal: Do AI systems generalize to visual software domains? In *International Conference on Learning Representations*, 2025.
- [21] Daoguang Zan, Zhirong Huang, Ailun Yu, Shaoxin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu, Dezhi Ran, Muhan Zeng, Bo Shen, Pan Bian, Guangtai Liang, Bei Guan, Pengjie Huang, Tao Xie, Yongji Wang, and Qianxiang Wang. SWE-bench-java: A GitHub issue resolving benchmark for Java. *arXiv preprint arXiv:2408.14354*, 2024.
- [22] Muhammad Shihab Rashid, Christian Bock, Yuan Zhuang, Alexander Buchholz, Tim Esler, Simon Valentin, Luca Franceschi, Martin Wistuba, Prabhu Teja Sivaprasad, Woo Jung Kim, Anoop Deoras, Giovanni Zappella, and Laurent Callot. SWE-polybench: A multi-language benchmark for repository level evaluation of coding agents. *arXiv preprint arXiv:2504.08703*, 2025.
- [23] Linhao Zhang, Daoguang Zan, Quanshun Yang, Zhirong Huang, Dong Chen, Bo Shen, Tianyu Liu, Yongshun Gong, Pengjie Huang, Xudong Lu, Guangtai Liang, Lizhen Cui, and Qianxiang Wang. CodeV: Issue resolving with visual data. In *Findings of ACL*, pages 7350–7361, 2025.
- [24] Shaoxiong Zhan, Shi Hu, Boyu Feng, Hai Lin, Andrew Gong, Zhengda Zhou, Jiaying Zhou, Yunyun Hou, Hao Su, and Hai-Tao Zheng. MM-IssueLoc: A controlled benchmark for evaluating visual evidence in multimodal repository-level issue localization. *arXiv preprint arXiv:2607.15205*, 2026.
- [25] Dongfu Jiang, Xuan He, Huaye Zeng, Cong Wei, Max Ku, Qian Liu, and Wenhui Chen. MANTIS: Interleaved multi-image instruction tuning. *Transactions on Machine Learning Research*, 2024.
- [26] Bingchen Zhao, Yongshuo Zong, Letian Zhang, and Timothy Hospedales. Benchmarking multi-image understanding in vision and language models. *arXiv preprint arXiv:2406.12742*, 2024.
- [27] Fei Wang, Xingyu Fu, James Y. Huang, Zekun Li, Qin Liu, et al. MuirBench: A comprehensive benchmark for robust multi-image understanding. In *International Conference on Learning Representations*, 2025.
- [28] Daoguang Zan, Zhirong Huang, Wei Liu, et al. Multi-SWE-bench: A multilingual benchmark for issue resolving. *arXiv preprint arXiv:2504.02605*, 2025.
- [29] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. In *Advances in Neural Information Processing Systems*, 2025.
- [30] Sergio Castro, Coen De Roover, Andy Kellens, Angela Lozano, Kim Mens, and Theo D’Hondt. Diagnosing and correcting design inconsistencies in source code with logical abduction. *Science of Computer Programming*, 76:1113–1129, 2011.
- [31] Raymond Reiter. A theory of diagnosis from first principles. *Artificial Intelligence*, 32:57–95, 1987.
- [32] Lianghong Guo, Wei Tao, Runhan Jiang, Yanlin Wang, Jiachi Chen, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, and Zibin Zheng. OmniGIRL: A multilingual and multimodal benchmark for GitHub issue resolution. *Proceedings of the ACM on Software Engineering*, 2025.

- [33] John Yang, Carlos E. Jimenez, Alexander Wettig, Shunyu Yao, and Shun Narayanan. Swe-bench multimodal: Do ai systems generalize to visual software engineering tasks? In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [34] Xue Wang et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, 2025.
- [35] Chunqiu Steven Xia, Yihan Deng, et al. Agentless: Demystifying llm-based software engineering agents. In *International Conference on Learning Representations*, 2025.
- [36] Zhen Pan et al. Swe-gym: Training software engineering agents with real-world tasks. In *International Conference on Learning Representations*, 2025.
- [37] Tianyang Liu et al. Repobench: Benchmarking repository-level code auto-completion systems. In *International Conference on Learning Representations Workshop*, 2023.
- [38] Yang Liu et al. G-eval: Nlg evaluation using gpt-4 with better human alignment. *Proceedings of EMNLP*, 2023.
- [39] Marco Tulio Ribeiro, Tong Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of nlp models with checklist. *ACL*, 2020.
- [40] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 2023.
- [41] SLSA. Supply-chain levels for software artifacts specification, 2023.
- [42] National Institute of Standards and Technology. Secure software development framework version 1.1, 2022.
- [43] Jean-Louis Letouzey. The sqale method for managing technical debt. *Managing Technical Debt*, 2012.
- [44] Niels Mundler, Mark Niklas Muller, Jingxuan He, and Martin Vechev. Swt-bench: Testing and validating real-world bug-fixes with code agents. *arXiv preprint arXiv:2406.12952*, 2024.
- [45] Linghao Zhang, Shilin He, Chaoyun Zhang, et al. Swe-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025.
- [46] Fan Cui, Hongyuan Hou, Zizhang Luo, Chenyun Yin, and Yun Liang. Hwe-bench: Benchmarking llm agents on real-world hardware bug repair tasks. *arXiv preprint arXiv:2604.14709*, 2026.
- [47] Jiahong Xiang, Wenxiao He, Xihua Wang, Hongliang Tian, and Yuqun Zhang. Evaluating and improving automated repository-level rust issue resolution with llm-based agents. *arXiv preprint arXiv:2602.22764*, 2026.
- [48] Ion George Dinu, Marian Cristian Mihaescu, and Traian Rebedea. Smellbench: Evaluating llm agents on architectural code smell repair. *arXiv preprint arXiv:2605.07001*, 2026.
- [49] Benjamin Miller et al. Repoexec: Evaluate code generation with repository-level executable benchmark. *arXiv preprint arXiv:2406.03095*, 2024.
- [50] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021.
- [51] Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language models. In *arXiv preprint arXiv:2108.07732*, 2021.
- [52] Fang Zhang, Bei Chen, Yue Zhang, et al. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Empirical Methods in Natural Language Processing*, 2023.
- [53] Xinyun Chen, Maxwell Lin, Nathanael Scharli, and Denny Zhou. Self-debugging: Teaching large language models to debug their own code. *arXiv preprint arXiv:2304.05128*, 2023.
- [54] Noah Shinn, Federico Cassano, Ash Berman, et al. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- [55] Zhou Zheng et al. Sweet: A framework for evaluating code agents with software engineering tasks. *arXiv preprint arXiv:2405.16203*, 2024.

- [56] Fernando Martinez-Plumed et al. Position: Benchmarking machine learning systems is a sociotechnical problem. *Proceedings of Machine Learning Research*, 2021.
- [57] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.
- [58] Giovanni Grano, Raffaele Ventura, and Harald C. Gall. On the prevalence of flaky tests in open source software. In *2018 IEEE International Conference on Software Maintenance and Evolution*, 2018.
- [59] Qiguang Chen, Chengyu Luan, Jiajun Wu, Qiming Yu, Yi Yang, et al. Omibench: Benchmarking olympiad-level multi-image reasoning in large vision-language model. *arXiv preprint arXiv:2604.20806*, 2026.
- [60] Wendong Bu, Yang Wu, Qifan Yu, et al. What limits virtual agent application? omnibench: A scalable multi-dimensional benchmark for essential virtual agent capabilities. In *International Conference on Machine Learning*, 2025.

A. Dataset Construction Details

The appendix is divided into a small number of top-level modules following the organization of benchmark papers such as OMIBench. Appendix A covers dataset construction and instance records; Appendix B gives the detailed main experiment and agent lifecycle; Appendix C defines evaluation and scoring; Appendix D reports detailed analysis and case studies; and Appendix E documents reproducibility, release, and the data dictionary. Auxiliary experiments and statistical templates are kept as subsections of the relevant modules rather than promoted to independent appendices.

A.1. Artifacts and accounting

Each instance is identified by repository, base commit, split, license, baseline scores, verification status, and gate-strength label. A complete model-instance record contains the unified patch, agent output, base evidence, treated evidence, verification result, budget record, and score record. Missing artifacts should be represented in a manifest with an explicit reason.

The evaluator distinguishes infrastructure failure, environment or dependency failure, missing or invalid patch, behavior regression, preserved behavior with no governance improvement, and preserved behavior with positive improvement. Behavior-broken runs are excluded from valid NGI aggregation but remain in failure statistics. Monetary cost is reported only when provider accounting is available; otherwise it is marked unavailable.

A.2. Rubric anchors

Scores of 1–2 indicate absent or non-functional governance, 3 indicates partial coverage with material gaps, 4 indicates an executable and reviewable standard, and 5 indicates broad coverage with evidence of maintainability and low regression risk.

Table 5: Compact rubric anchors used for each governance dimension.

Score	Interpretation
1	Absent, misleading, or actively broken practice
2	Minimal artifact with major gaps or unreliable execution
3	Partial coverage; useful in common cases but material gaps remain
4	Executable, reviewable, and appropriate for the repository
5	Broad, durable coverage with evidence of maintainability

A.3. Current release status

The current release is suitable for a research draft and descriptive analysis. Before external publication, private evaluation artifacts must be removed from the public package, public traces must receive a completeness manifest, and all model-instance results must record evaluator and container versions.

A.4. Instance Records and Behavior Gates

A.4.1. Instance schema

Table 6 lists the minimum fields required for a reproducible instance. Fields are separated into task, evidence, and result records so that a missing trace cannot be mistaken for a zero score.

Table 6: Minimum instance schema for a future public release.

Record	Required fields
Task	repository, base commit, split, language, license, applicability mask, task text
Environment	image or lockfile identifier, installation command, tool versions, timeout policy
Evidence	probe name, command, exit code, stdout/stderr digest, timestamp, applicability
Behavior	characterization tests, mutation sample, gate-strength label, pre/post status
Result	patch digest, changed files, score vector, NGI, breakage label, budget record
Provenance	evaluator version, prompt version, scaffold, model identifier, artifact completeness

A.4.2. Detailed gate-strength accounting

The current 60-instance release contains 24 instances with mutation-demonstrated gates, 27 with blind gates, 3 with vacuous gates, and 6 without a usable behavior gate. This distribution motivates reporting gate strength as a first-class variable. Future tables should report results for the full set, the mutation-demonstrated subset, and the no-gate subset separately.

Table 7: Behavior-gate categories and their interpretation.

Category	Count	Interpretation
Detected	24	Mutation is caught by the characterization gate
Blind	27	Mutation is not caught; risk evidence is weak
Vacuous	3	Gate executes but has insufficient discriminative power
None	6	No usable behavior gate is available

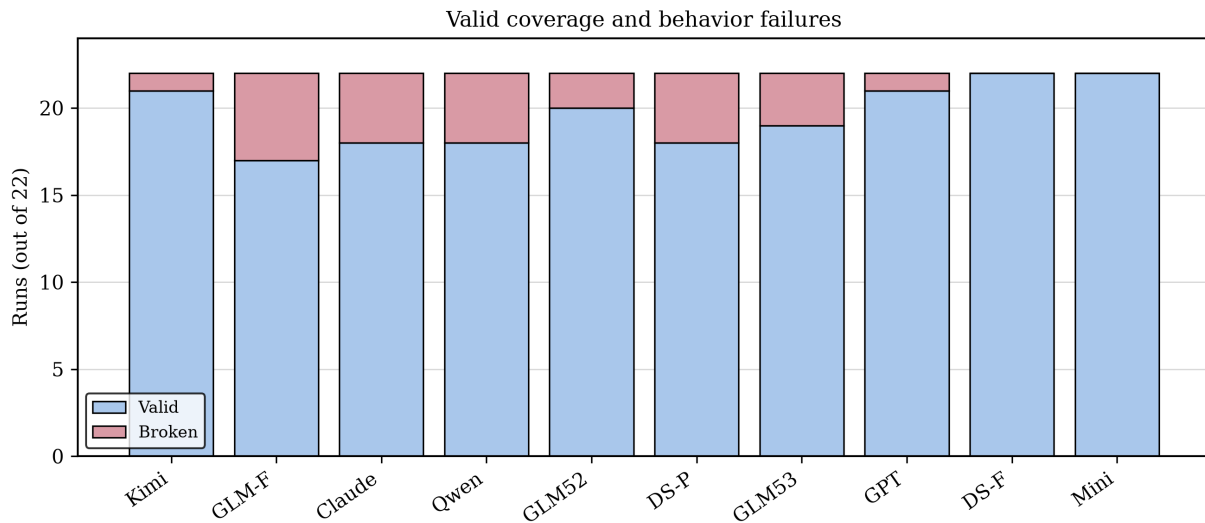


Figure 3: Coverage and behavior validity by model on the current public result records. Each bar is decomposed into valid and behavior-broken instances out of the same 22-instance pool, making denominator differences visible before comparing NGI.

A.4.3. Reproducibility checklist

The next artifact release should include: (1) a manifest of repository commits and licenses; (2) a container or environment lock for every public instance; (3) deterministic scorer configuration; (4) command-level logs and exit codes; (5) patch and trace completeness checks; (6) a separation policy for private tasks and verification evidence; (7) evaluator and judge version identifiers; (8) a script that reconstructs every main-paper table; and (9) a changelog for repository, rubric, or scoring updates.

A.4.4. Auxiliary experiment worksheet

For every planned experiment in Table 3, the final appendix should record the sampling frame, random seed, model and scaffold, time/token budget, excluded runs, paired comparison unit, confidence interval method, and exact command used to reproduce the table. We recommend filling the worksheet before looking at results so that analysis choices do not drift toward favorable outcomes.

A.4.5. Expanded rubric prompts

For D1, evaluators should ask whether tests exercise important behavior and whether CI runs the relevant tests on a clean checkout. For D2, they should ask whether the quality command is meaningful, scoped, and enforced rather than merely configured. For D3, they should ask whether a new contributor can install, use, modify, and report security issues. For D4, they should ask whether changes reduce unnecessary coupling and preserve understandable boundaries. For D5, they should ask whether another user can reproduce installation and execution from the declared environment. For D6, they should ask whether dependencies are controlled and whether security checks produce inspectable evidence. In every dimension, a file's presence without successful execution is insufficient for a score of 4 or 5.

A.4.6. Temporary auxiliary-results placeholder

The following checklist is intentionally left in the draft so that the later experimental package can be inserted without changing the paper structure.

Table 8: Fields to replace after the auxiliary experiments are delivered.

Artifact	Current draft content	Later replacement
10-repository manifest	Fixed stratified batch, single model	Actual repository IDs and split
Main rerun	Predicted NGI range 0.40–0.65	Observed per-instance scores
No-op baseline	Expected NGI 0.00	Measured scorer stability
Mechanical baseline	Expected NGI 0.10–0.25	Actual patch and evidence results
Rule baseline	Expected NGI 0.15–0.30	Actual deterministic baseline results
Gate ablation	Expected inflation 0.05–0.15	Gated/ungated paired statistics
Repeated seeds	Expected SD 0.08–0.18	Mean, SD, and confidence intervals
Qualitative cases	Planned high/medium/low examples	Actual patch excerpts and audit labels

All values in this subsection are hypotheses for planning and must not be cited as empirical evidence. The final version should retain the experimental protocol, replace the estimates with measured values, and add a machine-readable result file containing the exact 10-instance selection and run metadata.

A.5. Candidate Dataset and Quality Control

This appendix expands the construction description in the style of recent benchmark papers that separate the benchmark object, the construction funnel, the evaluation protocol, and the quality-control evidence. It is intentionally explicit about what is fixed, what is hidden, and what remains to be completed.

A.5.1. Dataset construction overview

The current dataset60 was constructed as a repository-governance benchmark rather than sampled from a synthetic checklist. The construction unit is a repository at a fixed commit. The final package contains 60 instances, divided into 22 public instances with shared model coverage and 38 private instances reserved for internal evaluation. The public/private distinction is applied after task validation and does not change the scoring definition.

The construction workflow is:

1. discover repositories with plausible engineering-health gaps;
2. freeze a reconstructable base commit and collect project metadata;
3. run baseline probes and assign the six-dimension applicability mask;
4. construct or validate a behavior characterization gate;
5. execute a dry-run of installation, probes, and scoring;
6. record artifacts, provenance, and exclusion reasons;
7. assign the instance to the public or private split and freeze the manifest.

The workflow intentionally separates discovery from inclusion. A repository can be useful for finding candidates because it lacks a workflow or has sparse tests, but it enters the final dataset only after its behavior, environment, and governance headroom are independently checked.

Table 9: Dataset-construction stages and concrete outputs.

Stage	Operation	Output
Discovery	Search active open-source repositories for governance signals	Candidate repository list
Pinning	Select a reachable pre-treatment commit	Repository and commit manifest
Baseline	Run installation, tests, quality, dependency, and security probes	Base evidence and score vector
Applicability	Decide which of D1–D6 are meaningful	Applicability mask and rationale
Behavior	Validate characterization tests and mutation sensitivity	Verification patch and gate-strength label
Dry run	Rebuild the environment and execute the full evaluator	Environment and evaluator log
Packaging	Hash artifacts and remove protected material from public outputs	Instance package and release manifest

A.5.2. Construction principles

SWE-Prometheus treats a repository as a maintained artifact rather than a collection of isolated code snippets. The benchmark therefore follows four principles. First, each task starts from a reconstructable repository snapshot. Second, the objective is open-ended enough to admit multiple valid engineering interventions. Third, each claimed improvement is connected to a probe or a structured judgment. Fourth, behavior preservation is tested independently from governance improvement.

These principles distinguish three artifacts that can otherwise be conflated: the task defines the engineering objective, the agent patch represents the intervention, and the evaluator evidence determines whether the intervention is useful and safe. The developer or construction patch, when present, is a construction asset and is not exposed as an oracle to the agent.

A.5.3. Candidate discovery funnel

Candidate discovery proceeds in four stages: broad repository retrieval, automated mechanical screening, manual validity inspection, and evaluator dry runs. Broad retrieval favors recall and may use repository metadata, visible workflow gaps, test sparsity, packaging inconsistencies, documentation gaps, and dependency hygiene signals. Mechanical screening removes repositories that cannot be cloned or pinned. Manual inspection verifies that the project has a meaningful functional core and that its governance gaps are not artifacts of a broken checkout.

The final dry run reconstructs the base snapshot, executes all applicable probes, and records unavailable evidence. A candidate is rejected when the environment cannot be made reproducible, when the task requires private services, when no dimension has plausible headroom, or when behavior cannot be characterized at all. This funnel prevents the benchmark from equating “many missing files” with “high-value engineering work.”

A.5.4. Baseline assessment and headroom

For every retained candidate, the baseline evaluator runs before any model treatment. The baseline record contains the raw probe outputs, the normalized score vector, the set of applicable dimensions, and the reason for every score. Headroom is computed relative to the dimension ceiling, but high headroom alone is not sufficient for inclusion: a repository with a completely non-functional environment is not a meaningful governance task.

The baseline pass also identifies which commands are safe to run without network access and which dependencies are merely preinstalled in the evaluator image. This distinction is particularly important for D5 and D6. Declared project dependencies are audited as project artifacts; evaluator convenience tools are not counted as evidence that the repository itself is reproducible or secure.

A.5.5. Behavior-gate construction

Behavior gates are constructed from repository-native tests whenever possible. If the native suite is too broad, a small characterization patch targets stable public behavior and records the command used to execute it. The patch is validated on the base snapshot before any model treatment. We then assess gate strength by injecting behavior-preserving or behavior-changing mutations where the repository and language permit it. A gate is labeled detected only when the mutation is caught; a passing but mutation-blind test remains blind rather than being promoted to strong evidence.

This construction step is kept separate from governance scoring. The gate protects against destructive interventions, while the six-dimensional scorer measures engineering readiness. A strong gate therefore increases confidence in a treatment outcome but does not directly increase the treatment score.

A.5.6. Split construction and leakage control

The public split contains the instances used for shared comparison and trace inspection. The private split retains tasks, verification patches, and outcomes that should not be available for targeted optimization. Split assignment is recorded in the manifest, and private artifacts are excluded from public result aggregation. Future releases should additionally perform repository-family separation, near-duplicate scanning, and temporal contamination checks before publishing a new public split.

A.5.7. Instance packaging

An instance package contains task metadata, a repository identity, a base commit, a dimension applicability mask, base evidence, a verification asset, and evaluator configuration. Public packages may include task descriptions and public traces, but private verification patches, hidden test details, and private model outcomes remain excluded. Every package receives a stable identifier and a manifest digest.

Packaging is performed before model assignment. The task text does not disclose a target score, a preferred file, or a reference implementation. If a repository contains generated files or large assets, the package records whether they are retained, omitted, or reconstructed. Any omission that can affect execution is treated as a validity concern rather than an undocumented convenience.

Table 10: Candidate screening checklist used before an instance enters the benchmark.

Gate	Pass condition	Recorded evidence
Identity	Repository and base commit are uniquely identified	URL, commit hash, license
Buildability	Snapshot can be installed or inspected with declared limits	Environment log, failure reason
Functionality	Project contains meaningful runnable or testable behavior	Entry points and test sample
Applicability	At least two governance dimensions are relevant	Dimension mask
Headroom	Baseline is below the dimension ceiling without being pathological	Baseline score vector
Behavior	A characterization test or gate can be constructed	Patch, command, gate strength
Isolation	No private credential or construction artifact is required	Network and secret policy

A.5.8. Quality-control passes

We define three independent quality-control passes. The first checks package integrity: identifiers, commit reachability, file presence, manifest hashes, and license metadata. The second checks evaluator integrity: base evidence is reproducible, the characterization test runs on the base state, and the treated-state harness accepts a known valid patch. The third checks interpretation: the score record includes evidence, the behavior result is consistent with the logs, and missing data are labeled explicitly.

The passes are deliberately separated because a package can be technically executable but conceptually invalid, or conceptually sound but impossible to reproduce. The release checklist records pass, fail, or not-applicable for every pass and preserves the reason for every exception.

B. Detailed Main Experiment

B.1. Task tuple

We represent an instance as

$$T_i = (R_i, c_i, M_i, A_i, E_i, V_i),$$

where R_i is the repository, c_i the pinned base commit, M_i the task metadata and applicability mask, A_i the agent-facing instruction, E_i the executable environment, and V_i the verification package. The agent receives $(R_i, c_i, M_i, A_i, E_i)$ but not protected parts of V_i . A rollout produces a patch $P_{i,m}$ and an execution trace $\tau_{i,m}$ for model m .

The evaluator computes base evidence B_i and treated evidence $B_i(P_{i,m})$. For each dimension d , the scorer returns a score in $\{1, 2, 3, 4, 5\}$ or an explicit inapplicable value. The result record is therefore a tuple of evidence, scores, behavior status, and budget rather than only a scalar.

B.2. Evidence record

Every probe should be represented by the following fields: probe identifier, command, working directory, environment identifier, start and end time, exit code, timeout status, output digest, normalized result, applicability, and human-readable interpretation. The raw log may be stored separately, but the normalized record must be sufficient to reconstruct why a score was assigned.

Table 11: Evidence fields and their role in auditability.

Field	Purpose
Probe ID	Stable name for the check across evaluator versions
Command	Exact executable command, including relevant flags
Environment	Runtime, image, dependency, and tool version identifiers
Exit status	Distinguishes pass, failure, timeout, and unavailable execution
Output digest	Links the normalized record to the retained log
Applicability	Indicates whether the check is meaningful for this repository
Interpretation	Explains the mapping from evidence to rubric score

B.3. Applicability and missingness

An inapplicable dimension is not equivalent to a failed dimension. For example, a repository without a releasable package may not receive a packaging score, while a repository that declares a package but cannot install it receives negative evidence. The scorer therefore records applicability separately from outcome and prevents missing evidence from being converted into an unexplained zero.

Missingness is categorized as not applicable, unavailable due to environment, unavailable due to timeout, malformed artifact, or not yet collected. This coding is necessary for later sensitivity analyses. The final report will include both complete-case summaries and summaries that retain unavailable evidence as a separate outcome.

B.4. Normalization and aggregation

Raw dimension scores are retained alongside normalized improvement. For an instance with applicable dimensions D_i , the headroom-normalized score is computed only over defined dimensions. We report the number of dimensions contributing to each aggregate, the raw score sum, the treated score sum, and the normalized value. This avoids over-interpreting an identical NGI computed from very different numbers of applicable dimensions.

B.5. Agent Protocol and Run Lifecycle

B.5.1. Agent-facing instruction template

The following template describes the stable task-level contract. The actual task-specific metadata and repository path are filled by the runner. It does not prescribe which files to edit or which governance intervention to choose.

Inspect the provided repository and improve its engineering readiness across the applicable governance dimensions. Preserve existing behavior and public interfaces. Make useful changes for future users and maintainers. Use repository evidence to prioritize changes, verify important claims with executable checks, and submit the resulting patch. Do not use hidden evaluator assets or any reference patch.

The instruction is intentionally open-ended. The agent must discover the repository's build system, tests, documentation conventions, and dependency structure rather than receiving a checklist of missing files. The runner separately provides tool descriptions, time limits, and submission semantics.

B.5.2. Run lifecycle

Each run follows the lifecycle below:

1. reconstruct the pinned base snapshot;
2. verify base environment and characterization behavior;
3. launch the model with the fixed task instruction;
4. archive the model output and candidate patch;
5. apply the patch to an independent clean snapshot;
6. run treated-state probes and behavior verification;
7. score the evidence and write the result manifest.

The agent-side container is discarded after submission. The evaluator does not reuse the agent's mutable working tree because doing so could allow unrecorded caches or generated files to influence the result.

B.5.3. Canonical-run policy

Retries are allowed for infrastructure failures such as a failed container launch, a transient model service error, or a verifier crash. A patch that is valid but incorrect is not retried merely to obtain a better result. When a retry occurs, the original record remains archived and the manifest identifies the canonical run with an explicit reason.

B.5.4. Budget accounting

Budget records contain model identifier, scaffold, maximum wall-clock time, actual elapsed time, input and output token counts when available, number of turns, tool calls, termination reason, and provider cost when available. A missing cost field is represented as unavailable. We do not compare providers using estimated prices from incomplete ledgers.

Table 12: Run-level budget fields for later auxiliary experiments.

Field	Definition
Model/scaffold	Exact model and execution framework
Wall time	Start-to-stop elapsed time and configured timeout
Turns/tools	Number of agent turns and external tool calls
Tokens	Input, output, and total tokens when exposed
Termination	Completed, timeout, service failure, invalid patch, or other reason
Cost	Provider-reported cost; unavailable otherwise

C. Evaluation and Scoring

C.1. Dimension-specific score anchors

The general 1–5 anchors are expanded below to reduce holistic scoring. A score of 4 requires executable or inspectable evidence appropriate to the repository; a configuration file by itself is not sufficient.

Table 13: Dimension-specific interpretation of an acceptable score.

Dim.	Score 3	Score 4–5
D1	Tests cover selected behavior but important paths or CI execution are missing	Important behavior is exercised in a clean, repeatable test/CI path
D2	Tool configuration exists but scope or enforcement is incomplete	Quality checks run, target meaningful files, and expose failures
D3	Basic usage or contribution information exists with material omissions	A new maintainer can use, change, and report issues responsibly
D4	Organization is understandable locally but coupling or duplication remains	Boundaries and dependency direction are maintainable and justified
D5	Installation works only with undocumented assumptions	Clean setup and execution are reproducible from declared metadata
D6	Dependencies are declared but audit or update evidence is incomplete	Dependency and security posture are controlled and inspectable

C.2. Evidence sufficiency rules

For D1, a passing test command must be paired with evidence that the tests exercise meaningful behavior. For D2, the existence of a formatter or linter configuration does not establish that it runs on the project’s relevant files. For D3, prose is evaluated for actionability rather than length. For D4, a large refactor is not rewarded if it increases churn without improving boundaries. For D5, a lockfile is useful only when it agrees with the declared project metadata. For D6, a dependency report must distinguish project dependencies from tools preinstalled in the evaluator image.

C.3. No-regression rules

A treated repository receives a regression flag when characterization behavior fails, an important public interface changes unexpectedly, or the patch causes the evaluator to lose the ability to run applicable probes. A governance improvement cannot compensate for a confirmed behavior break in the primary valid-run aggregate. This asymmetric rule reflects the practical cost of damaging a working project while adding maintenance metadata.

C.4. Judge calibration worksheet

Before the final scoring pass, evaluators should independently score a calibration set spanning low, medium, and high baseline quality. The worksheet records score, evidence cited, uncertainty, and whether the evaluator would

request a maintainer review. Agreement is computed before adjudication; adjudication changes are retained as a separate field.

C.5. Auxiliary Experiment Design

C.5.1. Fixed 10-repository batch

All auxiliary experiments will use one frozen batch of ten repositories. The batch should be selected before looking at auxiliary outcomes and should be accompanied by a manifest containing repository, commit, language, size band, base mean, applicable dimensions, gate strength, and public/private status. No experiment may silently substitute an easier repository after execution begins.

Table 14: Required fields for the frozen auxiliary batch manifest.

Field	Requirement
Instance ID	Stable repository-level identifier
Commit	Exact base revision used by every condition
Strata	Size, baseline-quality, and gate-strength bins
Applicability	D1–D6 mask and reason for exclusions
Environment	Image or lockfile and evaluator version
Selection rationale	Why the instance represents the intended stratum

C.5.2. Condition matrix

The minimal condition matrix contains five rows: no-op, Kimi-K3 main run, mechanical retrofit, rule-based baseline, and behavior-gate ablation. All rows use the same ten repositories. The optional repeated-seed row adds two more Kimi-K3 runs on a smaller five-repository subset if compute is limited.

Table 15: Unified condition matrix for the next experimental package.

Condition	Model	Repositories	Main comparison
No-op	None	10	Scoring and environment stability
Main	Kimi-K3/pi	10	Open-ended repository governance
Mechanical	None	10	Template-only intervention
Rule	Deterministic	10	Checklist-oriented automation
Gate ablation	Re-score	10	Effect of behavior filtering
Repeated seed	Kimi-K3/pi	5 or 10	Within-model variance

C.5.3. Pre-registered hypotheses

H1: the model treatment achieves higher NGI than the two non-semantic baselines but exhibits larger variance. H2: the behavior gate reduces apparent success by removing unsafe patches. H3: detected gates yield higher no-regression confidence than blind or vacuous gates. H4: improvement is concentrated in D1–D3, while D4–D6 require more repository-specific reasoning. H5: broad source edits correlate with behavior risk more strongly than targeted governance edits.

These hypotheses are directional and should be evaluated with paired instance-level records. If an estimate is not supported, the final paper will report the negative result rather than modify the hypothesis after seeing the data.

C.6. Statistical Analysis and Reporting Templates

C.6.1. Primary estimands

The primary estimand is the paired difference in NGI between the Kimi-K3 treatment and each baseline on the same ten repositories. Secondary estimands are the paired difference in raw dimension score, behavior-breakage rate, no-regression rate, verified-claim rate, and resource usage. We report per-instance values before any mean or median aggregation.

C.6.2. Mean–median reporting rationale

The main result table reports mean NGI as its single aggregate improvement measure. This choice keeps the leaderboard compact and makes the quantity directly interpretable as average normalized governance improvement across valid runs. Because the current public comparison contains 22 instances and one rollout per model-instance pair, the mean is still descriptive rather than an estimate of a stable population-level capability.

Median NGI is retained as a secondary diagnostic in the appendix rather than the main table. It is useful for identifying whether a model’s mean is driven by a small number of unusually large improvements: a mean substantially above the median indicates a right-skewed outcome distribution, while a close mean–median pair suggests more uniform instance-level behavior. The median is not used to replace the mean because it does not reflect the magnitude of improvements above or below the typical instance and can hide the contribution of a few high-value but valid interventions.

We therefore interpret the two statistics jointly when performing detailed analysis, but use only mean NGI in the compact main table. Any future claim about ranking stability will additionally report per-instance values, bootstrap intervals, valid coverage, and breakage rate. In particular, a high median cannot compensate for a high behavior-breakage rate, and a high mean cannot be interpreted as robust if it is supported by only a small valid denominator.

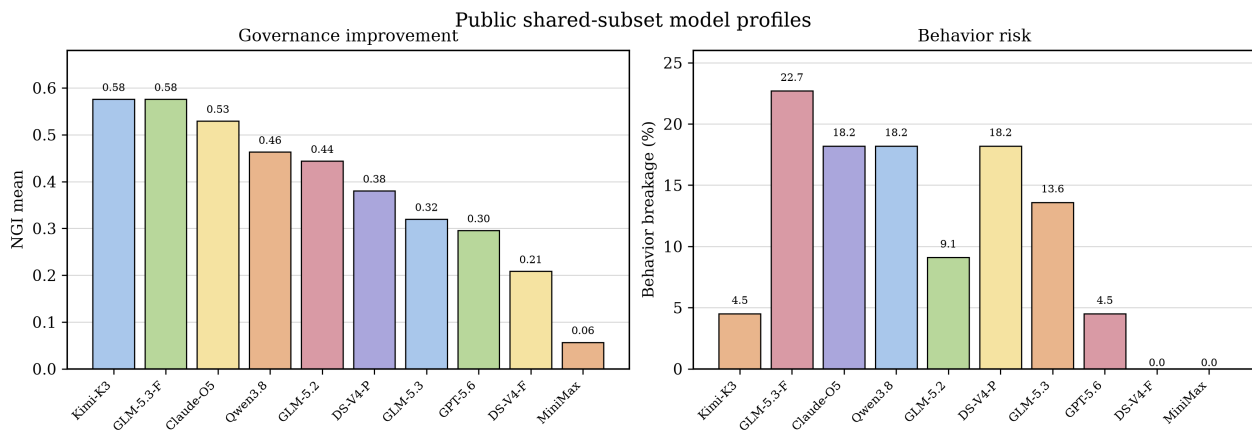


Figure 4: Model-level outcome profiles. The upper panel shows mean NGI on valid runs; the lower panel shows behavior-breakage rate over all 22 public instances. These panels motivate reporting improvement and safety together.

C.6.3. Uncertainty reporting

For ten repositories, asymptotic significance tests are not persuasive. We will therefore emphasize exact paired differences and bootstrap intervals over repository units, while clearly identifying the exploratory nature of the small batch. Repeated seeds, when available, add a second source of uncertainty and should be summarized with a hierarchical table separating between-repository and within-model variation.

C.6.4. Per-dimension reporting

For each condition, the final appendix should include a six-column D1–D6 table with base mean, treated mean, mean raw change, fraction improved, fraction unchanged, and fraction regressed. The table should also report the

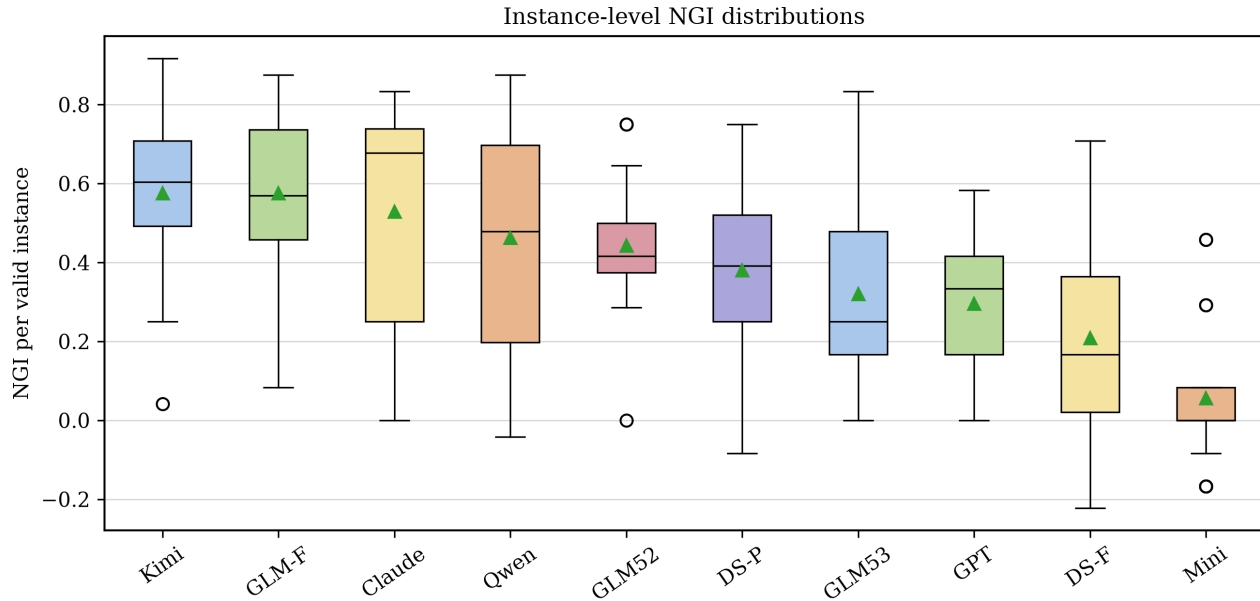


Figure 5: Instance-level NGI distributions for valid runs. Boxplots show the interquartile range and median, while the overlaid marker denotes the mean. The plot is a diagnostic companion to the mean-only main table and exposes skew and outlier sensitivity.

Table 16: Result table template to be populated after the auxiliary batch.

Condition	Valid	Broken	NGI mean	NGI median	No-reg.	Notes
No-op	TBD	TBD	TBD	TBD	TBD	scorer stability
Kimi-K3	TBD	TBD	TBD	TBD	TBD	primary treatment
Mechanical	TBD	TBD	TBD	TBD	TBD	template baseline
Rule	TBD	TBD	TBD	TBD	TBD	deterministic baseline
Gate-off	TBD	TBD	TBD	TBD	TBD	paired ablation

number of applicable instances per dimension. This prevents aggregate NGI from hiding a condition that improves one dimension while damaging another.

C.6.5. Missing and excluded runs

The analysis script must print a list of excluded runs with an exclusion code before generating any aggregate table. Codes include infrastructure, invalid patch, behavior broken, missing evidence, and inapplicable dimension. The paper should report both the denominator before exclusion and the denominator used for valid NGI.

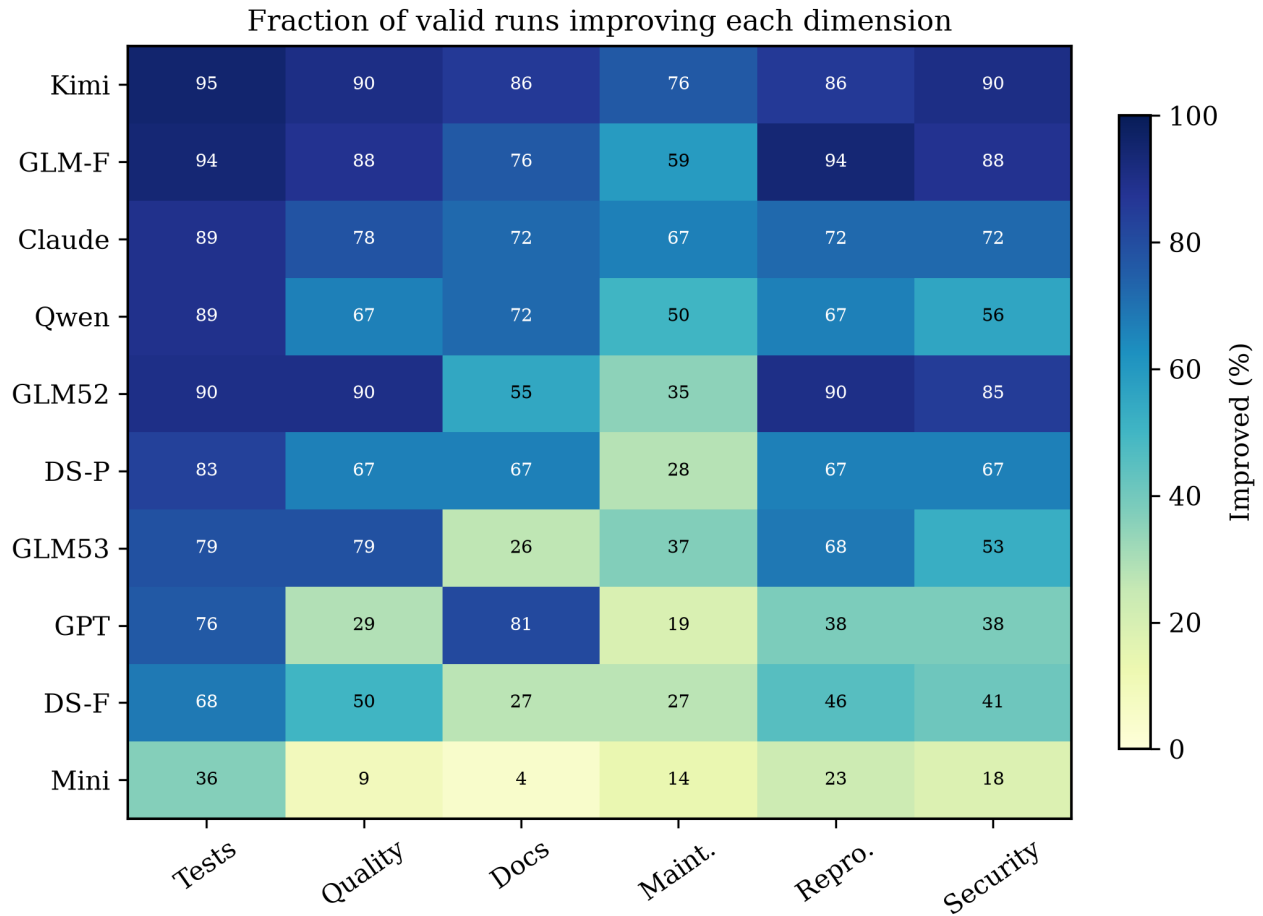


Figure 6: Per-dimension improvement coverage across the current model records. Each cell reports the fraction of valid runs that improved the corresponding governance dimension; the heatmap is intended to reveal whether aggregate gains are broad or concentrated in a small subset of dimensions.

D. Detailed Analysis and Case Studies

D.1. Case-study selection

We will select five cases after the auxiliary run: one high-NGI safe intervention, one high-NGI risky intervention, one low-NGI conservative intervention, one baseline-sensitive case, and one gate-strength disagreement. Selection will be performed from the frozen result table using predefined criteria, not by choosing only visually persuasive examples.

D.2. Case-study record

Each case study contains repository context, base governance profile, model diagnosis, changed-file map, executed verification, dimension-level score changes, behavior outcome, and an auditor’s assessment of whether the patch solved a meaningful maintenance problem. The developer or construction patch is not needed to explain a governance case and should remain hidden if it is part of protected evaluation.

D.3. Failure vignettes

For broken or low-value runs, the paper should show the earliest failure: environment misunderstanding, unsupported dependency change, ineffective test addition, vacuous CI, broad source churn, or insufficient verification.

Table 17: Qualitative case-study template.

Field	Content to provide later
Repository context	Purpose, ecosystem, size, and baseline governance gaps
Agent diagnosis	Evidence cited before editing
Intervention	Files and governance dimensions changed
Verification	Commands, outcomes, and behavior status
Score change	D1–D6 base and treated scores
Audit judgment	Useful, superficial, risky, or mixed, with rationale

These vignettes are important because the benchmark evaluates safe maintenance rather than only the presence of new files.

E. Reproducibility and Release

E.1. Directory-level release layout

The release should mirror the separation between public tasks, protected verification assets, results, and traces. Public artifacts must be sufficient to reconstruct the announced experiments but insufficient to reveal private outcomes or hidden answers. A manifest at the root maps every paper table to a script and every script output to a versioned input.

Table 18: Recommended release package layout.

Path	Content
tasks/	Public task metadata, commits, licenses, and applicability masks
environments/	Container definitions or environment locks
probes/	Versioned governance and behavior probe definitions
runs/	Canonical traces, patches, budgets, and evidence records
results/	Machine-readable per-instance and aggregate scores
analysis/	Scripts that regenerate paper tables and figures
docs/	Construction notes, scoring rubric, changelog, and limitations

E.2. Artifact completeness rules

An artifact is complete only when its manifest exists, its digest matches the stored file, and the downstream table can identify whether it was used. An empty directory is not treated as evidence of absence. If a run is interrupted, the manifest records the last completed stage and the reason for interruption.

E.3. Security and privacy

Repository licenses, credentials, private issue data, evaluator patches, and model-provider traces require separate review before release. Secret scanning is performed on artifacts and logs, but a positive scanner result is manually adjudicated because repository content may contain intentional examples. Private tasks remain excluded from public traces, score files, and aggregate tables.

E.4. Final paper update procedure

When the user supplies the auxiliary settings and results, the replacement process is: update the frozen batch manifest, replace every TBD field, regenerate tables from machine-readable results, revise the predicted-results paragraph into an observed-results paragraph, update the abstract and conclusion, and rerun the full PDF visual check. This keeps the current draft structurally complete while preserving a clean path to the empirical version.

E.5. Data Dictionary and Researcher Checklist

E.5.1. Core identifiers

The following identifiers should remain stable across the pilot and final releases. The instance identifier names a repository snapshot, the run identifier names one model execution, and the evidence identifier names one probe invocation. A score record must reference all three identifiers.

Table 19: Data dictionary for the benchmark records.

Name	Type	Definition
instance_id	string	Repository and pinned-snapshot identifier
run_id	string	Unique model/scaffold execution identifier
model	string	Exact model name or provider label
scaffold	string	Agent framework and runner version
base_commit	string	Commit used to reconstruct the untreated state
patch_digest	string	Digest of the submitted unified patch
behavior	enum	preserved, broken, invalid, or unavailable
gate_strength	enum	detected, blind, vacuous, or none
ngi	float	Headroom-normalized governance improvement
evidence_status	enum	pass, fail, timeout, unavailable, or not applicable

E.5.2. Researcher checklist before release

Before claiming an auxiliary result, confirm that the ten repositories were frozen in advance, every condition used the same base commit, no hidden evaluator asset was exposed, no-op results are stable, broken behavior runs are not silently dropped, score denominators are printed, and every aggregate number can be regenerated from the archived per-instance records. The final draft should also state which values were measured, which were predicted, and which remain unavailable.

E.5.3. Interpretation checklist

When comparing conditions, first check paired coverage, then behavior validity, then per-dimension change, and only then aggregate NGI. A higher aggregate score is not interpreted as an improvement if it is caused by a smaller denominator, a weaker gate, or a higher breakage rate. This ordering is especially important for the small ten-repository pilot, where one instance can materially affect the mean.